

TP2 - Adapter (Patrones Estructurales)

Materia: Metodología de Sistemas II

Unidad 2: Patrones de Diseño

Alumno: Nicolás Viruel

Lenguaje: Python (sin librerías externas)

1. Problema identificado

El sistema (40 archivos) usa `OldGeoService.get_location(ip)` que retorna `{ lat, lng, city, country }`.

Se migró a `NewGeoProvider.locate(ip)` con estructura distinta (`.coordinates.latitude`, `.address.locality`, etc.).

No se puede reescribir los 40 archivos ni modificar el proveedor externo.

2. Diagrama UML - Cuatro piezas

1. Cliente (40 archivos del sistema - código existente)
 - Llama a: `geo.get_location(ip)`
 - Espera recibir: `{ lat, lng, city, country }`
2. `OldGeoService` (interfaz que el cliente conoce)
 - Método: `get_location(ip) → dict`
3. `GeoServiceAdapter` (único archivo nuevo)
 - Implementa la misma interfaz que `OldGeoService`
 - Compone internamente a `NewGeoProvider`
 - Traduce `locate(ip)` al formato legacy de `get_location(ip)`
4. `NewGeoProvider` (proveedor externo - adaptee)
 - Método: `locate(ip)` con estructura distinta (`.coordinates`, `.address`)

Relaciones:

- Cliente → `OldGeoService` (depende de esta API)
- `GeoServiceAdapter` → `NewGeoProvider` (delega y traduce)
- Cliente NO conoce `NewGeoProvider` (transparencia del adapter)

3. Código cliente - SIN MODIFICAR

Este código representa los 40 archivos del sistema. No fue alterado.

```
def cliente_sistema(geo, ip):  
    data = geo.get_location(ip)  
    print(f"{data['city']}, lat={data['lat']}")
```

Antes (configuración):

```
geo = OldGeoService()  
cliente_sistema(geo, "200.45.123.10")
```

Después (único cambio en toda la app):

```
geo = GeoServiceAdapter() # <-- unico cambio  
cliente_sistema(geo, "200.45.123.10") # cliente identico
```

La función `cliente_sistema` no cambia ni una línea. Solo cambia que objeto se inyecta.

4. OldGeoService - SIN MODIFICAR

```
class OldGeoService:
    def get_location(self, ip):
        return {
            "lat": -34.6037,
            "lng": -58.3816,
            "city": "Buenos Aires (legacy)",
            "country": "AR",
        }
```

5. NewGeoProvider - SIN MODIFICAR

```
class NewGeoProvider:
    def locate(self, ip):
        # Retorna estructura incompatible con OldGeoService
        return LocateResult(
            coordinates=Coordinates(latitude=-34.6037, longitude=-58.3816),
            address=Address(locality="Buenos Aires", nation="Argentina"),
        )
```

6. GeoServiceAdapter - UNICO ARCHIVO NUEVO PERMITIDO

"""

TP2 - Adapter | Metodología de Sistemas II
Alumno: Nicolás Viruel

Unico archivo nuevo permitido: GeoServiceAdapter.
OldGeoService, NewGeoProvider y el cliente no se modifican.

Si llega un tercer proveedor (FutureGeoProvider):
crear FutureGeoAdapter(OldGeoService) y cambiar solo la línea de inyección.
Los 40 archivos cliente siguen igual.

"""

```
class GeoServiceAdapter(OldGeoService):
    def __init__(self):
        self._provider = NewGeoProvider()

    def get_location(self, ip):
        result = self._provider.locate(ip)

        # Traduce respuesta nueva al formato legacy
        return {
            "lat": result.coordinates.latitude,
            "lng": result.coordinates.longitude,
            "city": result.address.locality,
            "country": result.address.nation,
        }
```

Por fuera: expone `get_location(ip)` idéntico a `OldGeoService`.
Por dentro: delega en `NewGeoProvider.locate()` y traduce la respuesta.

7. Evidencia: cliente no modificado

Salida con OldGeoService (antes):

Buenos Aires (legacy), lat=-34.6037

Salida con GeoServiceAdapter (despues):

Buenos Aires, lat=-34.6037

Misma llamada cliente_sistema(geo, ip). Misma firma get_location(ip). Mismo acceso data["city"], data["lat"]. El cliente no sabe que cambio el proveedor.

8. Interrogante: tercer proveedor

Pregunta: Que tendria que cambiar si llega un tercer proveedor mañana?

Respuesta:

1. Implementar FutureGeoProvider con su API propia (sin tocar cliente ni OldGeoService).
2. Crear FutureGeoAdapter(OldGeoService) que traduzca la respuesta de ese proveedor al diccionario { lat, lng, city, country }.
3. Cambiar solo la linea de inyeccion en configuracion: geo = FutureGeoAdapter().
4. Los 40 archivos cliente siguen llamando get_location(ip) sin cambios.

El adapter es invisible: el sistema sigue creyendo que habla con la API vieja.

9. Justificación del patrón

Adapter resuelve incompatibilidad de interfaces sin refactor masivo. Reescribir 40 archivos es inviable; modificar el proveedor externo tampoco es opción. El adaptador de objetos (composición) envuelve al nuevo proveedor y expone la API legacy. Una Fachada unificaría subsistemas complejos; acá solo hay que traducir una respuesta.